

Software Architecture and Design Specification

Web Based ML Model Evaluator

Project: Web Based ML Model Evaluator

Version: 1.0

Team: Team 5

Authors: Denis Mangla (PES2UG24CS151), R P Pranav (PES2UG24CS917), Chellamani Adithya Iyer (PES2UG24CS129), Disha Bose (PES2UG24CS160)

Course: Software Engineering

Date: 15-09-2026

Status: Final – Ready for Submission

Revision History

Version	Date	Author	Change Summary
1.0	15-09-2026	Team 5	Initial architecture and design specification derived from SRS v1.1

Approvals

Role	Name	Signature/Date
Team Lead / Backend & Architecture	Denis Mangla	
Frontend & Results/Reports	R P Pranav	
Data Pipeline Engineer	Chellamani Adithya Iyer	
ML Engineer	Disha Bose	
Course Coordinator		

1. Introduction

1.1 Purpose

This document specifies the architecture and design of the Web Based ML Model Evaluator. It translates the requirements stated in SRS v1.1 into a component structure, a set of design decisions with their rationale, interface definitions and a security architecture, so that implementation can proceed without reinterpreting the requirements.

1.2 Scope

Covers the architecture and design of the web application through which a user uploads a tabular dataset, configures a preprocessing pipeline, selects and trains a supervised model, and compares evaluation metrics across completed runs. It also covers administration of user accounts and the dataset catalogue, and the audit trail behind both.

Deep-learning workloads requiring GPU acceleration, unsupervised learning, non-tabular datasets, automated feature engineering and the publishing of trained models as inference endpoints remain outside scope, consistent with SRS §1.2.

1.3 Audience

The four developers of Team 5, the course coordinator and evaluators for the Software Engineering course, and any future maintainer of the system.

1.4 Definitions

- ADR — Architecture Decision Record.
- ORM — Object-Relational Mapper; the Django data-access layer used throughout.
- Run — one execution of a selected algorithm over a prepared dataset, with its stored configuration and metrics.
- Pipeline — the ordered set of preprocessing operations applied before training.
- STRIDE — threat classification: Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege.
- TLS — Transport Layer Security.
- RTM — Requirements Traceability Matrix.

2. Document Overview

2.1 How to use this document

Section 3 states the architecture: the goals it must satisfy, the component structure, the pattern chosen and why, the risks accepted, and the security architecture. Section 4 states the design: the interaction flows, the interfaces between components, and the cross-cutting conventions for errors, logging and user experience. Requirement identifiers of the form MLE-F-###, MLE-NF-### and MLE-SR-### refer throughout to SRS v1.1 and are not restated here; section 3.8 maps them onto the components that satisfy them.

2.2 Related Documents

- Software Requirements Specification (SRS) v1.1 — the source of every requirement identifier used here.
- Software Test Plan (STP) v1.0 — defines the test cases referenced by the acceptance criteria.
- Requirements Traceability Matrix — Appendix of the SRS; extended in §3.8 with component references.

3. Architecture

3.1 Goals & Constraints

Goals. The architecture must make a training run reproducible from its stored configuration; keep the interface responsive while a model trains; isolate one user's data and one user's failures from another's; and stay small enough that four students can build, test and demonstrate it within one semester.

Constraints. Training executes on CPU only. An upload is limited to 50 MB and a run to 10 minutes of wall-clock time. Only supervised classification and regression on tabular data are supported. The system must run on a single ordinary development machine, with no cluster or managed cloud service assumed. Only permissively licensed open-source libraries may be used.

3.2 Stakeholders & Concerns

Stakeholder	Primary concerns
Registered User	Reaching a model comparison quickly; trusting that reported metrics are correct and reproducible; not losing work when a run fails.
Administrator	Acting on accounts and the dataset catalogue without needing to read the contents of another user's datasets.

Stakeholder	Primary concerns
Course evaluators	Traceability from requirement to component to test case; evidence that design decisions were reasoned rather than defaulted.
Developers (Team 5)	Clear module boundaries so four people can work in parallel without colliding; testable units.
Future maintainer	Understanding why the architecture is as it is before changing it.

3.3 Component (UML) Diagram

The diagram below presents the system as four UML subsystems: the Django Web UI, the Django application backend, the relational database, and server-side file storage. The backend subsystem contains the feature-specific components and the internal TrainingWorker. Dashed arrows represent dependencies or service-access relationships, directed from the component or subsystem requiring a service toward the component or subsystem providing it.

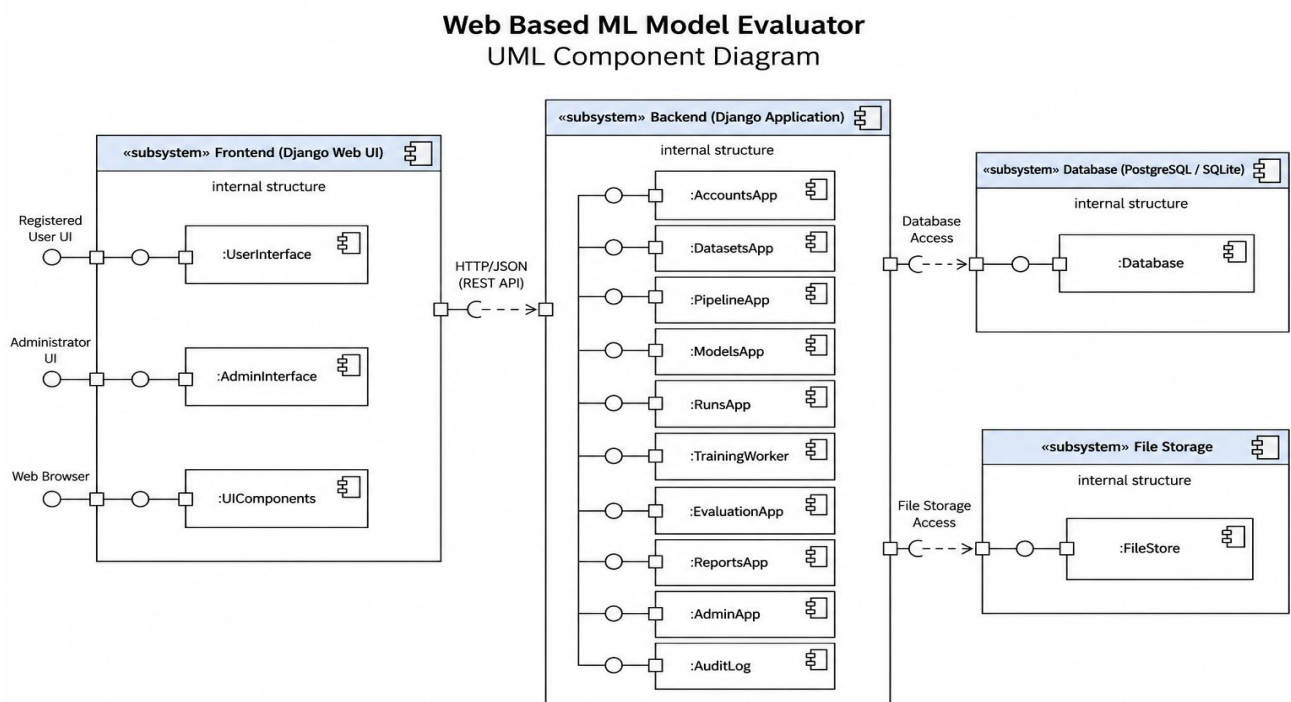


Figure 1 — UML component diagram of the Web Based ML Model Evaluator

3.4 Component Descriptions

Component	Responsibility
WebUI	Renders every screen through Django templates with Bootstrap, and draws metric charts with Chart.js. Polls a small JSON endpoint for job state so the page updates without a manual refresh. Holds no business logic.
AccountsApp	Registration, authentication, session lifetime and the Registered User / Administrator role distinction. Owns MLE-F-001 to F-003 and MLE-SR-002.
DatasetsApp	Accepts an upload, validates extension, size and parsed structure, writes the file to the per-user directory, profiles each column, and enforces dataset ownership. Owns MLE-F-010 to F-013 and MLE-SR-004.
PipelineApp	Target and feature selection, missing-value strategies, categorical encoding, numeric scaling and the seeded train/test split. Owns MLE-F-020 to F-023.

Component	Responsibility
ModelsApp	The catalogue of supervised algorithms and their hyperparameter definitions, ranges and defaults. Owns MLE-F-030 to F-032.
RunsApp	Creates a TrainingJob record, hands it to the worker, and exposes job state to the interface. Owns MLE-F-040 to F-042.
TrainingWorker	Executes a queued run in a background thread: loads the dataset, applies the stored pipeline, fits the model, requests metrics and persists the result. Enforces the 10-minute wall-clock limit of MLE-SR-005.
EvaluationApp	Computes the metrics appropriate to the problem type and assembles the side-by-side comparison of completed runs. Owns MLE-F-050 to F-052.
ReportsApp	Generates the evaluation report and exports a comparison as CSV. Owns MLE-F-060 and F-061.
AdminApp	Administrator pages for listing, deactivating and reactivating accounts and for managing the dataset catalogue. Owns MLE-F-062 and F-063.
AuditLog	Writes an append-only record of actor, action, resource, UTC timestamp and outcome for login, upload, deletion and administrative actions. Owns MLE-SR-006.
Database	Users, datasets and their profiles, pipeline configurations, runs, metrics and audit records. SQLite during development, PostgreSQL when deployed, reached only through the Django ORM.
FileStore	Uploaded dataset files, held under a per-user directory on the server file system.

3.5 Chosen Architecture Pattern and Rationale

A three-layer architecture was chosen: presentation, business and data. Each layer depends only on the layer beneath it. The business layer is divided into Django apps along the boundaries of the SRS feature groups, so that each requirement group maps to a primary component or a small set of cooperating components, allowing the four developers to work in parallel without editing the same modules. TrainingWorker and AuditLog sit in the business layer as well: the worker executes business logic away from the request path, and the audit component is a cross-cutting business concern rather than a store in its own right.

Microservices were rejected. The system runs on a single machine for a single class of user at a scale of tens of datasets; splitting it into separately deployed services would add network calls, service discovery and distributed failure modes while removing none of the actual complexity, and it would make the 10-minute demo harder rather than easier.

An event-driven architecture was also rejected. Only one asynchronous interaction exists in the system — starting a training run — and it is adequately served by a job record plus a worker thread. Introducing a message broker for a single event type would add an operational dependency that contributes nothing the job table does not already provide.

Within the layered structure, training is deliberately kept out of the request/response path. If a run executed synchronously, a 120-second training job would hold an HTTP connection open and block the interface, which would violate MLE-F-040 and make MLE-NF-001 unachievable.

3.6 Technology Stack & Data Stores

Layer / Concern	Technology
Presentation	Django templates, Bootstrap 5, Chart.js
Business	Python 3.11, Django; one Django app per feature group

Layer / Concern	Technology
Data & modelling	pandas for dataframe handling; scikit-learn for all estimators and metrics
Background execution (business layer)	Python threading with a TrainingJob table carrying state, timing and error reason
Relational store	SQLite in development, PostgreSQL when deployed; access solely through the Django ORM so the change requires no application code
File store	Local file system, one directory per user account
Transport	HTTPS with TLS 1.2 or higher on any deployed instance
Build & quality	pytest with coverage for unit and integration testing; Git and GitHub for source control and code review; Jira for backlog and sprint tracking.

3.7 Risks & Mitigations

Risk	Mitigation
A long-running or runaway training job consumes the machine and blocks other users.	The worker enforces a 10-minute wall-clock limit and the 50 MB upload cap bounds the input; breaching either terminates only the offending job (MLE-SR-005, MLE-NF-003).
A worker thread dies and leaves a run stuck in Running forever.	Job state is held in the database rather than in memory; a job whose worker terminates is marked Failed with a reason instead of being left indefinite.
Metrics are silently wrong, which is far worse than an obvious crash in a tool whose purpose is comparison.	Metrics are computed only by scikit-learn, never by hand, and unit tests assert them against reference values within 1e-6 (TC-EV-01, TC-EV-02).
A malicious upload executes code on the server.	Extension, size and parsed structure are validated before any write, and no user-supplied pickle or joblib artefact is ever loaded (MLE-SR-004).
A run cannot be reproduced when an evaluator asks for it during the demo.	The random seed and the complete pipeline configuration are persisted with every run (MLE-F-023, MLE-F-042).
In-process threading proves insufficient if the team later needs parallel runs.	The RunsApp interface is defined in terms of enqueue and status, so the worker can be replaced by Celery without changing its callers.
Four developers block on one another near the deadline.	Component boundaries follow the SRS feature groups, so each member owns a distinct set of modules.

3.8 Traceability to Requirements

Each requirement group is realised by the component named below; the test cases are those defined in the SRS and detailed in the Test Plan.

Requirements	Description	Component	Test cases
MLE-F-001 – F-003	Registration, authentication, roles	AccountsApp	TC-ACC-01 – 03
MLE-F-010 – F-013	Upload, validate, profile, ownership	DatasetsApp	TC-DS-01 – 04
MLE-F-020 – F-023	Target selection, imputation, encoding, split	PipelineApp	TC-PP-01 – 04
MLE-F-030 – F-032	Algorithm catalogue and hyperparameters	ModelsApp	TC-MS-01 – 03
MLE-F-040 – F-042	Background execution, job state, reproducibility	RunsApp, TrainingWorker	TC-TR-01 – 03
MLE-F-050 – F-052	Metrics and run comparison	EvaluationApp	TC-EV-01 – 03
MLE-F-060, F-061	Evaluation report and CSV export	ReportsApp	TC-RP-01, TC-RP-02
MLE-F-062, F-063	Account and dataset-catalogue administration	AdminApp	TC-AD-01, TC-AD-02
MLE-NF-001, NF-002	Response time and training time	WebUI, TrainingWorker	TC-PERF-01, TC-PERF-02
MLE-NF-003	Failure isolation	RunsApp, TrainingWorker	TC-REL-01

Requirements	Description	Component	Test cases
MLE-NF-004	Usability and keyboard reachability	WebUI	TC-UX-01
MLE-NF-005	Unit-test coverage of core modules	PipelineApp, ModelsApp, EvaluationApp	TC-MNT-01
MLE-NF-006	Retention of runs and audit records	Database, AuditLog	TC-OPS-01
MLE-SR-001	TLS in transit	Deployment configuration	TC-SEC-01
MLE-SR-002	Password hashing	AccountsApp	TC-SEC-02
MLE-SR-003	Server-side role and ownership checks	All application components	TC-SEC-03
MLE-SR-004	Safe handling of uploaded files	DatasetsApp	TC-SEC-04
MLE-SR-005	Job resource limits	TrainingWorker	TC-SEC-05
MLE-SR-006	Append-only audit records	AuditLog	TC-SEC-06

3.9 Security Architecture

Threats were classified with STRIDE and each is answered by a control already required by the SRS, so that the security architecture and the security requirements cannot drift apart.

STRIDE category	Threat in this system	Control	Requirement
Spoofing	An unauthenticated visitor reaches a dataset, run or report by guessing a URL.	Session authentication enforced server-side on every protected view.	MLE-F-002, MLE-SR-003
Tampering	A user edits a stored run or its metrics to improve a comparison.	Runs and metrics are written only by the worker; the application exposes no route that modifies a completed run.	MLE-F-042, SO-2
Repudiation	An administrative deletion cannot later be attributed to anyone.	Append-only audit record of actor, action, resource, UTC timestamp and outcome.	MLE-SR-006
Information disclosure	One user reads another user's dataset by supplying its identifier.	Ownership checked on the server for every read or write of a dataset, run or report.	MLE-F-013, MLE-SR-003
Information disclosure	Credentials or dataset contents leak through logs or transport.	TLS 1.2+ in transit; passwords stored only as salted one-way hashes; no credential material written to logs.	MLE-SR-001, MLE-SR-002
Denial of service	An oversized upload or an unbounded training job exhausts the shared machine.	50 MB upload cap; 10-minute wall-clock limit; breach terminates only the offending job.	MLE-SR-005, MLE-NF-003
Elevation of privilege	A Registered User reaches an administrator page.	Role checked on the server for every administrative view, independently of anything hidden in the browser.	MLE-F-003, MLE-SR-003
Elevation of privilege	A crafted upload executes code during deserialisation.	Extension, size and structure validated before any write; user-supplied pickle and joblib artefacts are never loaded.	MLE-SR-004

4. Design

4.1 Design Overview

The design follows the three-layer boundaries of §3.5. A request enters through WebUI, which holds no business logic; the application component that owns the relevant feature group performs the work; persistence happens through the ORM and the per-user file directory. The single asynchronous path is training: RunsApp records a job and returns immediately, TrainingWorker executes it, and WebUI polls for state. Every component that changes state writes to AuditLog, which no application route can modify.

Two flows are shown as sequence diagrams below: bringing a dataset into the system, and executing a run through to displayed metrics. Together they exercise every layer and both of the security-critical paths — file validation on the way in, and resource limiting during execution.

4.2 UML Sequence Diagrams

Figure 2 covers a dataset upload. Validation happens before any write to FileStore, so a rejected file never reaches storage. Profiling produces the column summary that the preprocessing stage later depends on.

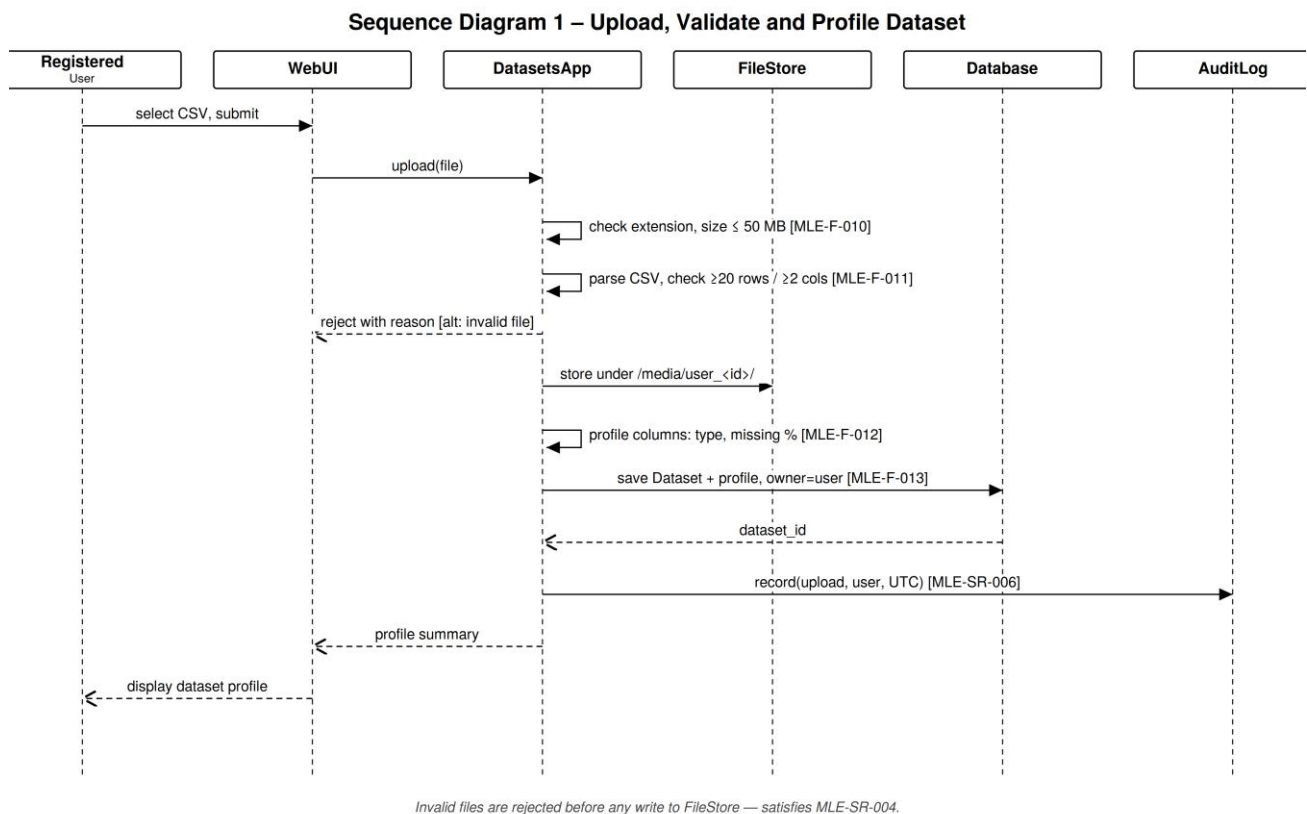


Figure 2 — Sequence diagram: upload, validate and profile a dataset

Figure 3 covers a training run. RunsApp returns a job key without waiting for the run to finish, which is what keeps the interface responsive; WebUI then polls for state at intervals of no more than five seconds until the run completes or fails.

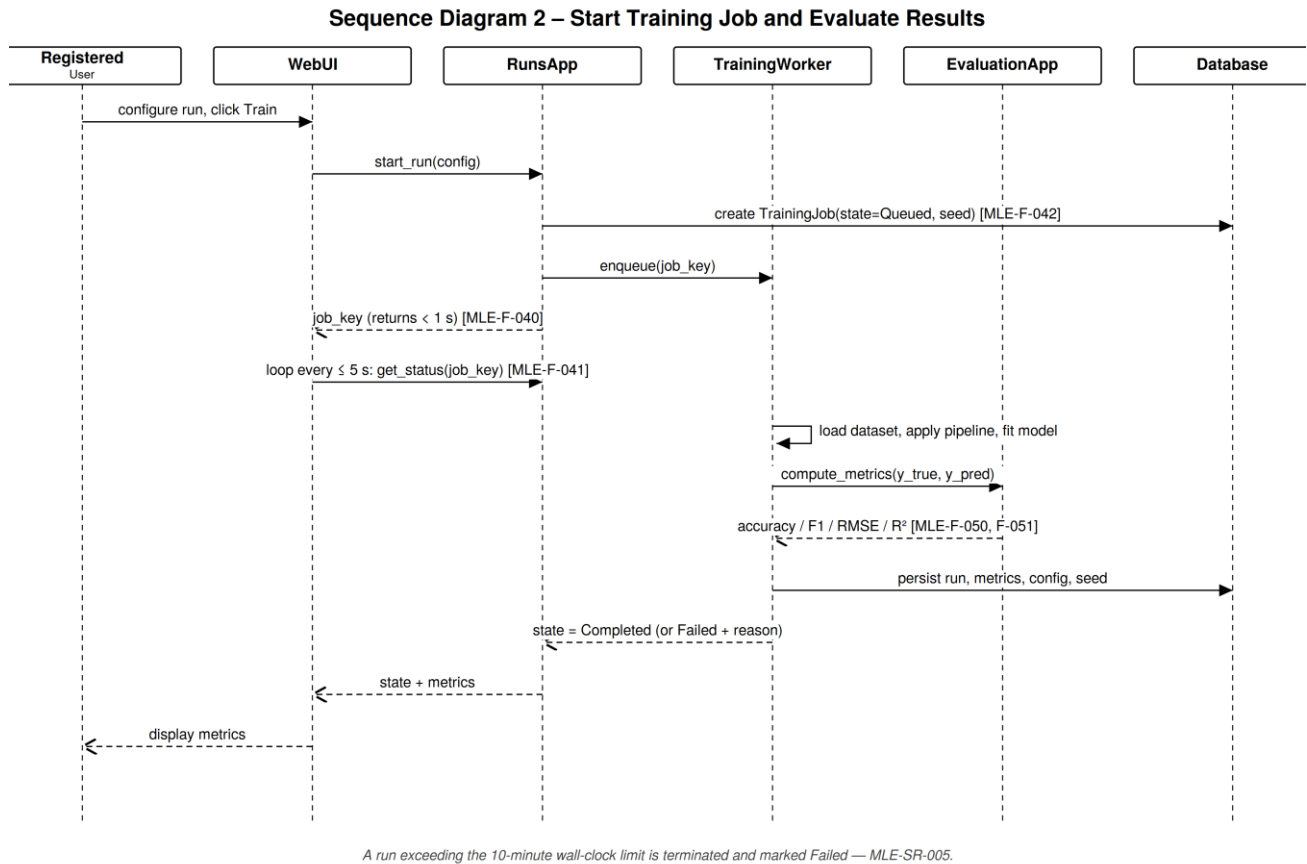


Figure 3 — Sequence diagram: start a training job and evaluate results

4.3 API Design

The application is server-rendered, so most interaction is form posts returning HTML rather than a public REST API. Two interfaces are nonetheless defined explicitly: the JSON endpoint that WebUI polls for job state, and the internal service interface between RunsApp and TrainingWorker. Documenting these two matters because they are the contract across the asynchronous boundary, which is where the design is easiest to get wrong.

Interface 1 — Job status endpoint (WebUI → RunsApp).

GET /api/runs/<job_key>/status

Auth: session cookie; server verifies the caller owns the run

Response: { "job_key": "r-8f3c", "state": "Running",
"started_at": "2026-09-09T11:04:22Z",
"elapsed_s": 37, "error": null }

States: Queued | Running | Completed | Failed

Errors: 401 not authenticated, 403 not the owner, 404 unknown job key

Interface 2 — Run execution service (RunsApp → TrainingWorker).

enqueue(job_key, run_config) -> None

run_config = { dataset_id, target_column, feature_columns,
imputation, encoding, scaling, split_ratio,
random_seed, algorithm, hyperparameters }

Precondition: TrainingJob exists with state = Queued

Postcondition: state becomes Completed with metrics persisted,
or Failed with a human-readable reason

Limits: terminated at 600 s wall-clock (MLE-SR-005)


```
get_status(job_key) -> { state, elapsed_s, error }
```

The run configuration passed to enqueue is exactly what is persisted against the completed run, which is what makes MLE-F-042 reproducibility achievable: re-running a stored configuration with its recorded seed reproduces the same split, the same fit and the same metrics.

4.4 Error Handling, Logging & Monitoring

- Errors are reported with a standard HTTP status and a message the user can act on: 400 for a rejected upload with the reason stated, 401 unauthenticated, 403 for a role or ownership failure, 404 for an unknown resource, 500 for an unexpected server fault.
- A failed run is never silent. The failure reason is stored on the job record and displayed next to the run, so a user is never left watching a job that stopped progressing (MLE-F-041).
- Logs record the actor, the action, the resource and the outcome. They never contain passwords, session tokens or dataset contents — a dataset may itself be sensitive, so its rows are never written to a log line.
- Audit records are separate from application logs, are append-only, and carry UTC timestamps (MLE-SR-006, MLE-NF-006).
- Monitoring during the project period covers the number of runs failed in the last 24 hours, median training duration against the 120-second target, and unit-test coverage of the three core modules reported by the CI pipeline.

4.5 UX Design

- The primary workflow is linear — dataset, pipeline, model, run, results — with the current stage always visible, so a first-time user is never uncertain about what comes next (MLE-NF-004).
- Job state updates without a manual refresh; a run in progress shows its elapsed time rather than an indefinite spinner.
- Every chart is accompanied by the equivalent numeric table, so no information exists only inside a chart. Colour is never the sole carrier of meaning, and every form control carries a text label reachable by keyboard.
- Validation failures are explained in terms the user can act on — which rule failed and what to change — rather than as a generic rejection.
- Destructive administrative actions require explicit confirmation and are recorded in the audit trail.

4.6 Open Issues & Next Steps

- Replacing the in-process worker with Celery and Redis if parallel runs become necessary; the RunApp interface was defined to make this substitution possible without changing callers.
- ROC curve and confusion-matrix heat map as visual additions to the metrics already delivered.
- PDF rendering of the evaluation report, currently delivered as CSV export only.
- Cancellation of a queued or running job by its owner, which depends on a queue supporting revocation.
- An administrator dashboard reporting queue depth and recent failures, meaningful only once a distributed queue replaces the in-process worker.

5. Appendices

5.1 Glossary

ADR — Architecture Decision Record. ORM — Object-Relational Mapper. Run — one execution of an algorithm over a prepared dataset with its stored configuration and metrics. Pipeline — the ordered preprocessing operations applied before training. Seed — the recorded random value that makes a split and a fit reproducible. STRIDE —

Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege. TLS — Transport Layer Security. RTM — Requirements Traceability Matrix.

5.2 References

- Software Requirements Specification, Web Based ML Model Evaluator, v1.1, Team 5.
- Software Test Plan, Web Based ML Model Evaluator, v1.0, Team 5.
- IEEE 42010 — Systems and software engineering: architecture description.
- OWASP Top Ten — web application security risks.
- Django documentation — security, authentication and ORM.
- scikit-learn user guide — model selection and metrics.

5.3 Tools

- draw.io — UML component and sequence diagrams.
- Git and GitHub — version control and code review.
- Jira — backlog and sprint tracking.
- pytest with coverage — unit and integration testing.